

Informe Final del Proyecto CAGS – Project ARES

(Autonomous Repair Exploration System)

◆ Propietarios del Equipo

- **Juan Camilo Gómez Duarte** — Líder del proyecto y encargado del diseño, entrenamiento y dirección del backend de IA.
 - **Gustavo Adolfo Castillo Páez** — Responsable del diseño, ensamblaje y documentación técnica del dron CAGS_F413.
 - **Ángel Daniel Scarpetta Ruiz** — Desarrollador del backend administrativo y encargado del control y gestión del sistema de datos.
 - **Sara Sofía Alzate Charry** — Desarrolladora del frontend de control (interfaz NASA) y del panel de visualización principal.
-

◆ Descripción General del Proyecto

CAGS – Project ARES (Autonomous Repair Exploration System) es un sistema integral orientado a la **operación autónoma de drones especializados en inspección y reparación** en entornos críticos, como estaciones espaciales, zonas remotas o ambientes industriales peligrosos.

El ecosistema combina **inteligencia artificial, tecnología de comunicación en tiempo real, control web y procesamiento distribuido** para garantizar operaciones seguras, auditables y escalables.

◆ Objetivo General

Desarrollar una plataforma completa que integre **IA, backend, frontend y sistema de drones** para realizar misiones de diagnóstico y reparación remota de forma autónoma, con monitoreo humano en tiempo real.

◆ Arquitectura del Sistema

El sistema se compone de **cuatro capas principales**:

1. Capa Física / Dron (Gustavo Castillo)

- Hardware de vuelo y sensores.
- Sistema de navegación y control de telemetría.

- Captura de datos en tiempo real (temperatura, energía, posición, fallas).
- Enlace de comunicación con el backend vía **WebSocket** y **MQTT**.

2. Capa Lógica / Backend (Ángel Scarpetta)

- Implementada con **Django REST Framework (DRF)**.
- Gestiona datos de drones, telemetría, inventario, eventos, reparaciones y auditorías.
- Autenticación y autorización basada en **JWT** y roles.
- Control de acceso mediante clase de permisos **RolePermission**.
- Base de datos: **SQLite** (desarrollo) / **PostgreSQL** (producción).
- Seguridad con **Argon2**, **HTTPS**, cookies seguras y logs cifrados.

3. Capa de Interfaz / Frontend (Sara Alzate)

- Desarrollada con **React (Vite) + React-Bootstrap**.
- Comunicación en tiempo real con **WebSocket (Django Channels)**.
- Visualización de datos en panel tipo “Centro de Control NASA”.
- Módulos: Dashboard, Alarmas, Reparaciones, Inventario, Logs, Entrenamiento y Configuración.

4. Capa de Inteligencia Artificial (Juan Camilo Gómez Duarte)

- Backend de IA responsable de interpretación de datos de telemetría y predicción de fallas.
- Entrenamiento de modelos para diagnóstico autónomo y priorización de reparaciones.
- Comunicación bidireccional con el backend mediante **API REST** y **WebSockets**.
- Capacidad de aprendizaje continuo con datasets enviados desde el frontend.

◆ Roles y Responsabilidades

Integrante	Rol	Responsabilidades principales
Juan Camilo Gómez Duarte	Líder del Proyecto / IA Backend	Coordinación general, arquitectura de IA, control de flujos de datos, integración entre backend y modelos

Integrante	Rol	Responsabilidades principales
		de predicción.
Gustavo Adolfo Castillo Páez	Ingeniero de Drones	Diseño técnico del dron, documentación de sensores, sistemas de navegación, telemetría y ensamblaje físico.
Ángel Daniel Scarpetta Ruiz	Desarrollador Backend	Creación del backend administrativo (Django), gestión de roles, auditorías, telemetría y seguridad de datos.
Sara Sofía Alzate Charry	Desarrolladora Frontend	Creación de la interfaz gráfica para visualización y control, conexión con backend y diseño UX/UI.

◆ Módulos Principales

1. Módulo de Usuarios y Roles

- Roles: **admin**, **ingeniero**, **operador**.
- Permisos diferenciados por acceso:
 - *Admin*: control total del sistema.
 - *Ingeniero*: edición técnica (drones, eventos, inventario, reparaciones).
 - *Operador*: solo lectura.
- Seguridad basada en **JWT** con expiración corta y tokens de refresco.

2. Módulo de Drones / Telemetría

- Registra datos como número de serie, estado de conexión, batería, temperatura, versión de firmware y ubicación.
- Permite asociar reportes de daños con severidad, tipo y evidencia (foto/video).
- Comunicación en tiempo real con el backend para actualizaciones instantáneas.

3. Módulo de Reparaciones

- Registro detallado de cada intervención (fecha, materiales, estado, evidencia multimedia).
- Asociación con daños previos.

- Firma digital y hash de integridad para auditoría.

4. Módulo de Auditoría

- Registro de eventos críticos y anomalías del sistema.
- Logs inmutables con hash de verificación.
- Exportación de registros en JSON o CSV.

5. Módulo de Inventario

- Control de piezas, herramientas y materiales.
- Filtros por categoría, criticidad y disponibilidad.
- Control de stock con permisos diferenciados (solo ingenieros pueden modificar).

6. Módulo de Eventos / Alarmas

- Visualización en tiempo real de alarmas críticas.
- Sistema de notificación con niveles de prioridad.
- Historial y opción de “Acknowledge”.

7. Módulo de Entrenamiento y Órdenes

- Permite enviar órdenes de operación y entrenamiento a los drones.
- Subida de datasets y métricas de IA.
- Historial de sesiones de entrenamiento y rendimiento.

◆ Seguridad y Estándares

- **HTTPS/TLS obligatorio.**
- **MFA** para roles críticos.
- **Protección CSRF/XSS/Clickjacking** mediante políticas CSP y SameSite.
- **Cifrado en reposo** (DB encryption / field-level encryption).
- **Logs firmados digitalmente** para auditoría.
- **Backups periódicos** y plan de recuperación.
- **Evaluación OWASP Top 10** antes del despliegue.

◆ Integraciones Técnicas Clave

- **MQTT / RabbitMQ:** Canal de comunicación de baja latencia.
- **Swagger / OpenAPI:** Documentación interactiva de endpoints.
- **ELK Stack:** Sistema de almacenamiento y análisis de logs (Elasticsearch, Logstash, Kibana).
- **Django Channels + Socket.io:** Comunicación bidireccional en tiempo real.

◆ Estado Actual del Proyecto

- ✓ Backend completo y funcional, con endpoints organizados y autenticación segura.
- ✓ Frontend en desarrollo final, conectado al backend mediante WebSocket y API REST.
- ✓ Drones configurados y comunicando telemetría correctamente.
- ✓ IA entrenada en entorno local para diagnóstico predictivo y reparación.

◆ Conclusión General

CAGS – Project ARES representa un avance significativo en la integración de **inteligencia artificial, robótica y computación distribuida**, aplicada a la **exploración y reparación autónoma**.

El sistema combina precisión técnica, seguridad y modularidad, permitiendo adaptarse a múltiples entornos (espaciales, industriales o de rescate).

El proyecto consolida un flujo completo **IA ↔ Backend ↔ Frontend ↔ Dron**, preparado para pruebas operativas y futuras expansiones.

◆ Bibliografía y Referencias

- **NASA (National Aeronautics and Space Administration).** Documentación e inspiración visual sobre operaciones autónomas y telemetría de misiones espaciales.
- **Project Gemini Archives (NASA, 1965–1966).** Referencia histórica para el diseño y lógica de mantenimiento orbital automatizado.
- **OpenAI — ChatGPT (2025).** Asistencia en la estructuración y redacción técnica del informe.

- **xAI — Grok AI (2025).** Soporte en la optimización de algoritmos de código y estructuras lógicas del backend.
- **Google Gemini (2025).** Apoyo en la generación visual de materiales y simulaciones para pruebas de imagen.